

Secure Event-Driven User Registration Microservice

Role: Principal Solutions Architect

Difficulty: junior

Project Type: backend

Duration: 2 days

Focus: security

Candidate: Siddharth Rao

Project Overview

Build a secure, event-driven microservice architecture for user registration, demonstrating cloud-native design, domain-driven modeling, and basic security controls.

Technologies

- Python
- Kafka
- Docker
- Kubernetes
- Terraform
- AWS
- JWT
- HTTPS
- Domain-Driven Design
- Microservices
- Event-Driven Architecture

Requirements

- Implement a user registration service that publishes events to Kafka
- Implement an event consumer that processes registration events and stores user data
- Secure the API with JWT authentication and HTTPS/TLS
- Use Terraform to provision a local Kubernetes cluster (e.g., Minikube) or AWS free tier resources
- Containerize services with Docker
- Document the architecture, deployment steps, and security considerations

Objectives

- Demonstrate ability to design secure microservices
- Implement event-driven communication
- Apply cloud-native tools and Domain-Driven Design
- Showcase basic security controls in a backend system

Functional Requirements

- POST /register – accepts user details and returns JWT
- GET /users – returns list of registered users (JWT protected)
- Publish UserRegistered event to Kafka
- Consume UserRegistered event and persist user data
- Audit logging of registration events

Non-Functional Requirements

- Scalable and resilient design
- Secure communication (HTTPS/TLS)
- JWT-based authentication and role-based authorization
- Clear documentation and code comments
- Automated tests for core functionality

Modules

User Registration Service

Handles user sign-up, validates input, publishes events, and issues JWTs

- Input validation
- Password hashing
- JWT issuance
- Kafka producer
- HTTPS endpoint

Event Processor Service

Consumes registration events, persists user data, and logs audit records

- Kafka consumer
- Domain model mapping
- SQLite persistence
- Audit logging

Implementation Plan

Planning

- Define domain model (User, Event)
- Sketch architecture diagram
- Set up Git repo and folder structure

Development

- Implement User Registration Service in Python (FastAPI)
- Set up Dockerfile and Docker Compose
- Implement Event Processor Service
- Configure Kafka locally or via Docker

Testing

- Write unit tests for services
- Run integration tests for Kafka communication
- Perform security tests for JWT and HTTPS

Deployment

- Write Terraform scripts for local Minikube or AWS resources
- Deploy services to Kubernetes cluster
- Verify end-to-end flow and security

Evaluation Criteria

Correctness of Implementation (40%)

All functional requirements are met and services communicate correctly via Kafka

Security Measures (30%)

API uses HTTPS/TLS, JWT authentication, password hashing, and audit logging

Code Quality & Documentation (30%)

Clean code, meaningful comments, comprehensive README, and architecture diagram

Deliverables

- Source code repository
- Docker images
- Terraform scripts
- Architecture diagram
- README with deployment instructions

Suggested Folder Structure

- services/user-service
- services/processor-service
- infra/terraform
- docs
- scripts

API Endpoints

POST /register

Accepts user details, creates user, publishes event, returns JWT

GET /users

Returns list of registered users (JWT protected)

Database Design

users

Persist registered user data

- id
- username
- email
- password_hash
- created_at
- status

Testing Plan

Candidate Fit

The project leverages Siddharth's strengths in architecture, microservices, event-driven design, and cloud platforms while challenging him to apply security best practices in a concise, backend-focused implementation.